Chapter 05

Dialogs, Overlays, and User Feedback

4 lessons • 25 hr

elevify

CHAPTER 05 CONTENTS

Chapter 05

4 lessons • 25 hr

Chapter 05

4 lessons • 25 hr

Chapter 05 • Dialogs, Overlays, and User Feedback

Lesson 01 • AlertDialog and Confirmation Flows

27 pages • 41 min

elevify

Lesson 01 • AlertDialog and Confirmation Flows

41 min • 27 pages

◎ What you will learn

Builds modal AlertDialogs for confirmations and informational messages. Proper dialog management prevents common UI blocking bugs.

AlertDialog and Confirmation Flows

Summary

AlertDialog Basics: Creation and Open/Close

An AlertDialog is a floating overlay that grabs user attention for decisions. Create it with a title, content (any Flet control), and actions list of buttons. Opening requires adding the dialog to page.overlay, setting open=True, then calling page.update. To close, set open=False and update again. The dialog is hidden until explicitly opened. This simple pattern blocks the main app until the user responds, making it ideal for confirmations like delete or error acknowledgment.

Actions and Modal vs Dismissible Modes

Each button in the actions list gets an on_click handler. Cancel sets open=False and updates, confirm does the action then closes. Keep functions small. The modal property controls outside-click behavior. With modal=True, clicking outside does nothing — use for destructive actions. With modal=False (default), outside click closes the dialog, suitable for low-stakes prompts. Choosing the right mode matches the dialog's stakes and prevents accidental dismissals.

Async Handling Using asyncio.Event

To wait for a user decision without blocking the UI, use asyncio.Event. Create the event before opening the dialog. In the confirm callback, set a result variable and call event.set. Cancel does the same with false. In your async function, await event.wait(). The coroutine pauses, UI stays responsive, and execution resumes when the user clicks. This pattern is reusable: write an async function that returns bool, then call it from anywhere in your app.

Common Mistakes and When to Use Dialog

Common mistakes: forgetting page.update (nothing changes), not adding dialog to page.overlay before opening, not resetting open=True when reusing a dialog, and blocking the event loop inside callbacks. Use AlertDialog for decisions requiring a response. For passive info, use a snack bar. Modal dialogs are for irreversible actions like delete; dismissible ones for soft confirmations. Matching dialog type to stakes makes apps polished.

Content Flexibility and Next Steps

The content area accepts any Flet control: simple text, TextField for input, or icons and colors to reinforce tone. Structure stays the same. Next, learn about snack bars and banners — lighter feedback that informs without demanding a reply. Knowing when to use each feedback type matters as much as building them. These patterns handle virtually all confirmation scenarios in real apps, from deleting to navigating.

NOTES

AlertDialog and Confirmation Flows

Transcript

AlertDialog in Flet: Asking for Confirmation

Every app you build has that moment — you need to stop the user and ask a question. Delete this item? Confirm this action? That's exactly what AlertDialog does.

In this lesson, you'll learn to open and close dialogs, wire up actions with callbacks, pick between modal and dismissible modes, and handle async results cleanly. It's a short, focused conversation — you decide whether to block interaction until they respond.

What Is an AlertDialog?

An AlertDialog is a floating overlay that grabs the user's attention. It sits on top of your app and blocks everything else until the user makes a choice.

Use it when you need a decision before the app can proceed — like confirming a destructive action or acknowledging an error.

Creating a Dialog

Creating an AlertDialog in Flet takes just a few lines. For instance, to confirm a delete action, you'd write:

```
dlg = ft.AlertDialog(
title=ft.Text('Confirm Delete'),
content=ft.Text('This action cannot be undone.'),
actions=[ft.TextButton('Cancel'),
ft.TextButton('Delete')]
)
```

The title and content accept any Flet control — text, images, or even complex layouts. The actions list holds the reply buttons.

No dialog appears on screen yet. You've only defined it, like setting up a template.

Opening and Closing the Dialog

Opening a dialog takes three lines. First, you append the dialog to `page.overlay`. Then you set its `open` property to `True`. Then you call `page.update`.

To close it, you just set `open` to `False` and call `update` again. The overlay is a list where Flet keeps controls that float above the normal layout — that's for dialogs, bottom sheets, and banners.

Wiring Up Actions

Each button in the actions list is a regular Flet control — you attach `on_click` handlers just like any other. The Cancel button calls a function that sets `open` to `False` and updates the page.

The confirm button does the actual work first, then closes the dialog. Keep these functions small and focused: one function per

NOTES

AlertDialog and Confirmation Flows

Transcript

outcome. That keeps the flow easy to follow and straightforward to test.

Modal vs Dismissible Mode

Flet's `modal` property controls outside-click. Set `modal=True` to force an explicit choice — clicking outside does nothing. Use for destructive actions where accidental dismiss is costly.

With `modal=False` (default), clicking outside closes the dialog. That's fine for low-stakes prompts like quick confirmations.

Handling Async Results

When your app needs the user's answer before continuing, use an async pattern. The simplest way is an `asyncio.Event`. Create it before opening the dialog.

In the confirm callback, set a result variable and call `event.set`. Then `await event.wait` in your async function. The UI stays responsive while the coroutine pauses. Execution resumes the moment the user clicks.

Putting It All Together

Put it all together and the flow works like this. The user clicks a button, which calls your open dialog function. The dialog appears.

The user picks confirm or cancel. The matching callback fires, closes the dialog, and either executes the operation or discards it. That simple four-step sequence handles nearly every confirmation scenario you'll run into in real apps — from deleting to navigating.

Common Mistakes to Avoid

A few mistakes keep coming up. Forgetting `page.update` is the most common. Nothing changes on screen until you call it.

Second, not adding the dialog to `page.overlay` before setting open to `True`. Third, when reusing a dialog, set `open` back to `True` explicitly each time.

Fourth, never block the event loop inside a callback. If you need to wait, use `async` and `await`.

When to Use AlertDialog (and When Not To)

AlertDialog isn't always the answer. For passive info, reach for a snack bar. A dismissible dialog works for soft confirmations that are easy to undo.

Save modal dialogs for irreversible actions like deleting data or submitting final forms. Match the dialog to the stakes. That's what sets polished apps apart.

Flexibility in Content

NOTES

AlertDialog and Confirmation Flows

Transcript

Any Flet control fits inside an AlertDialog's content area — that's real flexibility. You can keep it simple with just a text message. Or embed a `TextField` to gather input before confirming.

For stronger signals, use icons and colors to reinforce the tone. A red warning icon signals a destructive action. The structure stays the same regardless of what you put inside.

Creating a Reusable Async Pattern

Here's the pattern. Write `async def ask_user(page, message)` returning `bool`. Inside, create an `asyncio.Event` and a result dict.

Confirm sets result to `True` and calls `event.set`. Cancel sets result to `False` and calls `event.set`. Await `event.wait`, close dialog, return result. Then anywhere: `confirmed = await ask_user(page, 'Delete this?')` and branch on the answer.

Key Points to Remember

Three points to remember. The open and close cycle: add to `page.overlay`, toggle `open`, and call `page.update`.

Choose `modal` for destructive actions, dismissible for low-stakes prompts. And for awaiting user input, `asyncio.Event` keeps things clean. These patterns handle virtually any confirmation scenario.

What's Next

You've got the patterns for confirmation dialogs. Next up: snack bars and banners — a lighter kind of feedback. They inform users without demanding a reply. Knowing when to use each matters just as much as knowing how to build them.

NOTES

Further study materials Lesson 1

Content type: Text

Using Flutter AlertDialog to Display Alerts with Material and Cupertino Styles

Available at https://medium.com/@antocara/como-crear-un-alertdialog-en-flutter-edca9eb602c

Full Content

Summary

What is an AlertDialog?

An AlertDialog is a Flutter component used to show alerts to users when events occur that need their attention. It works on both Android and iOS. For Android developers, it is similar to the Dialog class. For iOS developers, it is like the UIAlertController class. In Flutter, you can display an AlertDialog with either Material Design (Android-like) or Cupertino (iOS-like) style. Both handle alerts similarly.

Creating an AlertDialog: Two Steps

Creating an AlertDialog in Flutter involves two steps. First, you create the alert itself by setting its title, message, and buttons. Second, you show the alert to the user. The alert can be made using either the AlertDialog class for Material Design or CupertinoAlertDialog for iOS style. Both classes accept similar parameters, such as title, content, actions, and some platform-specific ones.

Parameters for AlertDialog

The AlertDialog and CupertinoAlertDialog classes have several parameters. Common ones include title (a Widget for the title), content (a Widget for the message), and actions (a list of Widgets for buttons). AlertDialog also has backgroundColor, shape (for rounded corners or borders), and elevation (for shadow effect). These platform-specific parameters are not available in CupertinoAlertDialog.

Displaying the AlertDialog

Once the alert is created, you show it by calling the showDialog function. This function requires the context and a WidgetBuilder, which returns the alert widget. The process is the same for both AlertDialog and CupertinoAlertDialog. The difference lies in the appearance: animations and background style match the native look of Android or iOS.

More Resources

For more content about mobile programming, including Flutter, Android, and Swift, you can visit the blog CompilacionMovil.com.

NOTES

Further study materials Lesson 1

Summary

The article there provides further details on creating alerts and other UI components. The examples given in the original text use GIFs to show code snippets for creating and displaying both types of dialogs.

NOTES

Further study materials Lesson 1

Transcript

How to Create an AlertDialog in Flutter

In most applications, both for Android and iOS, it is necessary to show alerts to users due to events occurring in the app that require special user attention.

This is done using a component called in Flutter **AlertDialog**. In the following lines, we'll see **how to create an AlertDialog in Flutter** and the different options it offers.

*[Image: https://miro.medium.com/1*OCekKGvIIZzEH5E0FQpsBQ.png]*

For those who have developed on Android, this functionality is the same as the Dialog class offers in its different forms: AlertDialog, DialogFragment, etc. For those more familiar with iOS, this functionality is provided by the UIAlertController class.

In Flutter, we can display the **AlertDialog** with the look of either the Android UI or the iOS UI; both forms exist, although the handling of the classes to use and the options are practically the same.

Creating an AlertDialog in Flutter

The **creation of the AlertDialog in Flutter** can be divided into two parts, or in other words, two steps are necessary:

- First, we create the alert itself, where we set the title, the message, and the buttons that will appear in the alert.
- Second, we invoke this alert and display it to the user.

The code for creating the first step would be as follows:

This is where we differentiate whether we want to create the alert with a Material Design (Android) aesthetic or an iOS aesthetic. There are two classes: AlertDialog and CupertinoAlertDialog.

These classes accept a series of parameters through their constructor, with which we configure the alert to display. Among the basic parameters are:

- `title`: This is a Widget where we set the title of the alert.
- `content`: Again, we have the option to add a Widget to create the content of our alert. Being a Widget opens up the possibility of including practically any content we want here.
- `actions`: This parameter accepts a list of Widgets and is where we put the buttons that the alert will show and the corresponding event we want to execute when any of them is pressed.
- `backgroundColor`: The name itself makes it clear what this parameter is for. This parameter is only applicable in the AlertDialog class, not in the Cupertino one.
- `shape`: Thanks to this parameter, we can customize the alert

NOTES

Further study materials Lesson 1

Transcript

with different options: curved borders, different types of border lines, thickness, etc. This parameter is only applicable in the AlertDialog class, not in the Cupertino one.

- `elevation`: To set an elevation on the z-axis for the alert. This parameter is only applicable in the AlertDialog class, not in the Cupertino one.

The first three are common parameters in both types of dialogs, and then depending on whether we want our app to have a look closer to iOS aesthetics or Android aesthetics, there will be a series of properties that are only compatible with one or the other. The documentation makes this quite clear.

Once we have created the alert, we only need to display it. To display it, we need to invoke the `showDialog` function. This function can be invoked from anywhere in our code and we pass it a series of necessary parameters, such as the context and a `WidgetBuilder`, which is simply a function that returns a Widget. That is exactly what we created in the previous step.

Let's see the code.

The difference between the first and the second case is the way the alert is displayed. The animations, the background gradient, are different, resembling how it is done natively in each operating system.

You can find more content about mobile programming on my blog, CompilacionMovil.com, where I usually publish content about Flutter, Android, and Swift.

NOTES

Further study materials Lesson 1

Content type: Text

Confirmation Dialogs as Intentional Friction to Prevent Irreversible User Errors

Available at https://uxplanet.org/confirmation-dialogs-how-to-design-dialogues-without-irritation-7b4cf2599956

Full Content

NOTES

Summary

Confirmation Dialogs: Purpose and Psychology

Confirmation dialogs are intentional interruptions that ask users for explicit consent before risky actions. They introduce friction to prevent costly mistakes, acting as a built-in risk management system. Human error accounts for up to 33% of data loss in SaaS, and studies show users make 3-6 errors per hour. These dialogs protect users by forcing a pause, shifting from fast to slow thinking, and building trust through thoughtful design.

Anatomy and When to Use Confirmation Dialogs

A well-designed confirmation dialog includes an icon, title, description, primary and secondary buttons, and optional friction. Use them for irreversible actions, actions with serious consequences, complex domino effects, rare actions, and explicit consent. Effective dialogs are rare and purposeful; overuse causes dialog fatigue. They are essential for actions like permanent deletion, financial transactions, or system changes where errors are catastrophic.

When Not to Use and Better Alternatives

Avoid confirmations for reversible, simple, low-risk, or frequent actions. Instead, use Undo patterns: the system performs the action immediately and shows a brief undo option. This reduces anxiety and encourages exploration. Examples: Gmail's undo after deleting, Google Photos. Undo builds trust by creating a safety net without interrupting workflow. For errors, use inline validation rather than modals.

Proactive UX and Best Practices

Proactive design eliminates unnecessary dialogs by separating dangerous buttons, requiring multiple steps, providing instant feedback, using safe defaults, and showing built-in warnings. Best practices for confirmations include clear titles, concise messages, action-based button labels, specific cancel labels, visual risk signals, minimal noise, and added friction for critical actions like typing 'DELETE' or holding a button.

Further study materials Lesson 1

Summary

Accessibility, Future, and Trust

Accessibility ensures all users can use dialogs: use role='alertdialog', aria labels, keyboard navigation with focus trapping, high contrast, and icons not relying on color. Future UX may use AI to predict errors and provide ambient feedback. The key takeaway: well-designed confirmations protect users without making them feel powerless. They build trust through thoughtful friction, preventing mistakes while respecting user time.

NOTES

Further study materials Lesson 1

Transcript

Confirmation Dialogs: How to Design Dialogs Without Irritation

*Confirmation dialogs are more than just pop-ups — they're moments of trust, friction, and protection. This article explores how to design them intentionally — when to use them (or not), how to avoid annoying users, and how to build smarter, safer interactions that prevent costly mistakes. We'll cover psychological insights, best practices, alternatives like undo, anatomy breakdowns, and real-world UX patterns. **And at the end — don't miss my personal design experiment and the final set of practical takeaways.** *

*[Image: https://miro.medium.com/1*9WRpBwN4lp5yiQfSuNQkAg.png]*

Almost everyone has encountered a situation where, after an automatic action in the interface, a window pops up asking **"Are you sure?"** At first glance, this looks like an annoying trifle, but in fact it is one of the most key patterns for protecting users from themselves. Confirmation dialogs are not just pop-ups; they are tools that interrupt the user's flow by introducing an intentional pause, helping them avoid mistakes and minimize the consequences of human error.

In this article, I want to figure out how these **digital "stop signals"** work, when they are absolutely necessary, and when they cause more problems than they solve. I will also explain why this is important and how to design them so that they work for trust rather than against it. Here is what we will explore:

Table of Contents

What Is a Confirmation Dialog, Really?

Essentially, a confirmation dialog box is an intentional interruption. It is a small element of the user interface designed to stop you and ask for your explicit consent before allowing an action that may be risky or irreversible. As Jakob Nielsen writes in **Confirmation**

NOTES

Further study materials Lesson 1

Transcript

Dialogs Can Prevent User Errors — If Not Overused:

> *" "A confirmation dialog: Asks users whether they are sure that they want to proceed with a command that they have just issued to a system."*

Think of it as a forced pause and rethink. It is not just decoration, not just an extra click — it is an intentional checkpoint built into the flow. Why? Because the consequences of skipping this pause can be catastrophic. Accidentally deleting an entire photo album, publishing a critical article before it is ready, or deleting important files with a single click — these are the kinds of unintended disasters that confirmation dialogs are designed to prevent. In many cases, they are the last line of defense. And that brings us to a very interesting design concept — **intentional friction.**

As **Anthony Ongaro** explains in his article **Why Intentional Friction Is a Game-Changer**:

> *" "Intentional friction is adding more difficulty around an easy action you want to do less of, so you have more space to make a conscious choice."*

Intentional friction is a purposeful UX strategy where small resistance or pauses are introduced into a flow to slow down interactions and encourage users to reconsider or reflect before proceeding. It differs from unintentional friction — it is not there to block users, but to protect them and enhance trust.

Here is a simple flow that illustrates how a confirmation dialog works in a typical interface — from the initial button click to the final outcome.

[Image: Confirmation dialog - User Flow]

The Psychology Behind Confirmation

Usually, designers want everything to be smooth, seamless, and frictionless. But sometimes an intentional pause, a little friction, is exactly what is needed. This is necessary to protect the user from what is called reckless intuitive behavior, especially when the stakes are really high.

Think of it as a **built-in risk management system**. Now for the numbers. The absence of this friction is very costly. The human factor — can you imagine? — accounts for approximately **30% of all disasters involving data loss**. And in a number of areas, this figure continues to grow.

> *" According to a report commissioned by Asigra and conducted by ESG (**SaaS Data Protection: A Work in Progress**), **accidental deletion accounts for 33% of all data loss incidents in SaaS applications**. Other major causes include account closure without*

NOTES

Further study materials Lesson 1

Transcript

data reassignment (29%), malicious deletion by an employee (23%), and schema misconfiguration (20%). The same report notes that 60% of business data is now stored in the cloud, and Gartner predicts that 70% of businesses will suffer unrecoverable SaaS data loss — emphasizing the critical need for both prevention and user-centered protection mechanisms.

The scale of these figures is enormous. And this is no exception: studies show that the **average frequency of human errors** is between three and six per hour — and this figure rises sharply if the interface is confusing or simply poorly designed. It is important to emphasize that this is not about blaming users. It is about creating systems that anticipate the likelihood of errors and prevent costly mistakes before they happen.

The Anatomy of a Confirmation Dialog

Well-designed confirmation dialogs are not just about asking **"Are you sure?"** — they are intentional, layered UI elements crafted to pause the user, provide clarity, and prevent irreversible mistakes. Below is a breakdown of the key components that make up an effective confirmation dialog:

1. **Icon (Visual Cue)** — Signals the severity or type of action (e.g., warning, danger, info).
2. **Title (Headline / Question)** — Clearly states the action being confirmed, phrased as a direct question.
3. **Description (Context / Consequence)** — Explains what will happen if the user confirms — highlight permanence and risks.
4. **Primary Action Button (Destructive CTA)** — Executes the destructive action.
5. **Secondary Action Button (Safe Exit / Cancel)** — Allows the user to opt out safely.
6. **Friction Mechanism (Optional)** — Adds a layer of intentional friction for high-risk actions.
7. **Signal Emphasis Ring (Optional)** — Draws visual attention to the icon using radiating rings or pulsing effect — reinforces severity without adding noise.

[Image: Anatomy of a Confirmation Dialog]

When to Use Confirmation Dialogs

And now for the main question: when should confirmation dialogs really be used? UX literature is unanimous on this point: with extreme caution. Their effectiveness depends directly on their rarity. The more often you display such dialogs, the faster they will become background noise and lose their power. So in what situations are they really justified?

NOTES

Further study materials Lesson 1

Transcript

1. Irreversible actions. Things that cannot be undone: complete deletion of files (not just to the trash, but forever), closing an account with all the associated information — photos, messages, profile.

2. Actions with serious consequences. Financial transactions, risk of confidential data leakage, critical system changes — for example, turning off the server or accidentally opening private information to everyone.

3. Complex actions with a "domino effect." When one action affects many related elements or users. For example: removing a participant from a team project or changing the primary payment method for a family subscription.

4. Rare actions. Things we do not do often and may not remember all the consequences of: publishing a major article or performing a mass data operation.

5. Explicit consent. Situations where the system needs your direct "yes": installing an update, connecting an external device, confirming a change in access rights.

When Confirmation Dialogs Are Unnecessary

We have already discussed when they are truly justified. But it is equally important to understand the opposite: when it is better not to use them. Excessive use leads to a phenomenon known in UX as **"dialog fatigue"**. If users are asked "Are you sure?" for every little thing, their reaction becomes predictable: they stop reading and automatically click "OK." As a result, the most important warnings lose their meaning and fail to fulfill their protective function. Where not to use:

1. Reversible actions. If the user can easily undo an operation, additional confirmation is redundant.
Example: In Google Drive, when a user moves a file to the Trash, the system does it immediately but shows a banner — File moved to Trash. [Undo]

2. Simple or low-risk actions. Actions that do not break workflows or cause harm do not need confirmation.
Example: In Notion, if a user toggles between light and dark mode or switches a text block from "Paragraph" to "Heading," it happens instantly — no confirmation required.

3. Frequent actions. Repeating confirmations for common tasks leads to dialog fatigue.
Example: In Slack, when you send a message or react with an emoji, the action is immediate — no "Are you sure you want to send?" popup. Messaging is a **high-frequency** action. Adding a confirmation would feel intrusive and slow.

4. Errors and validation. Input mistakes should be handled with inline messages, not modals.

NOTES

Further study materials Lesson 1

Transcript

Example: In Figma, when you enter an invalid email while inviting someone to a file, the system highlights the input field with an error message: "Please enter a valid email address."

Undo: A Better Alternative

What if the action is reversible? In such cases, confirmation can be replaced with a much more elegant pattern — **Undo**. The principle is simple:

? The system performs the action immediately.
? Then a small notification (banner or "toast") appears with a clear "Undo" button.
? The user has a few seconds to change their mind.

Examples:

1. *Gmail*: after deleting an email, a message appears saying "Conversation moved to trash. Undo."
2. *Google Photos*: when deleting a photo, a notification appears with the option to instantly undo the operation.

This approach does not interrupt the workflow and provides a sense of security. Undo has another important advantage: it reduces anxiety. Traditional "Are you sure?" prompts reinforce the fear of making a mistake and focus attention on the risks. Undo, on the other hand, creates the opposite experience — a feeling of safety net. Users feel more confident, try new scenarios, and experiment. Mistakes cease to be catastrophic and become a natural part of interaction. This builds trust and makes the product more user-friendly.

Proactive UX Instead of Confirmations

Sometimes the best confirmation dialogue is no dialogue at all. The modern approach to interface design — **proactive design** — tries to completely eliminate the need for such interruptions.

How does it work?

? Separation of dangerous actions. Critical buttons are physically separated from everyday ones.

? Several steps for serious operations. This prevents accidental pressing from starting the process.

? Instant feedback. The user immediately sees the result of the action and can react.

? Safe default values. The system assumes the most reliable scenario (like autocorrect in Word: it corrects, but leaves a chance to roll back).

NOTES

Further study materials Lesson 1

Transcript

? Built-in warnings. Important but non-critical information is displayed in the interface — as a tooltip or inline message, rather than in a pop-up window.

This design gives a sense of confidence: users are less afraid to try new things because they know that an accidental mistake will not be catastrophic.

Best Practices for Confirmation Dialogs

Confirmation dialogs should protect users — not irritate them. When designed well, they prevent mistakes, reduce regret, and build trust. Here is how to make them effective rather than annoying:

1. Use a Clear, Specific Title

Avoid generic titles like *"Are you sure?"* Instead, clearly state what the user is about to do. Users rely on mental shortcuts (heuristics) to process decisions quickly. A vague title creates cognitive load — users pause to interpret what is at stake. A direct question reduces ambiguity and speeds up comprehension.

[Image: Confirmation Dialogs: Use a Clear, Specific Title]

2. Keep the Message Concise and Informative

Use supporting text only when needed to explain the consequences or permanence of the action. Be brief, direct, and avoid legal jargon. If the action is irreversible, say so. Humans tend to scan rather than read, especially under time pressure. Too much text leads to information overload and decision fatigue. A short, focused message respects the user's limited attention and improves understanding of risk.

[Image: Confirmation Dialogs: Keep the Message Concise and Informative]

3. Use Clear, Action-Based Button Labels

Ambiguous options like "Yes/No" force users to pause and mentally map buttons to actions. This can lead to mistakes — especially in destructive scenarios. Descriptive labels reduce the chance of misclicks and improve confidence.

[Image: Confirmation Dialogs: Use Clear, Action-Based Button Labels]

4. Rethink the "Cancel" Button

Avoid using "Cancel" as a catch-all. When possible, label the safe exit clearly to reduce confusion and increase clarity. The word "Cancel" is semantically neutral — and in complex interfaces, it is often unclear what it will cancel. This uncertainty causes hesitation or avoidance behavior. Replacing it with context-aware labels helps

NOTES

Further study materials Lesson 1

Transcript

users feel more in control.

[Image: Confirmation Dialogs: Rethink the "Cancel" Button]

5. Use Visual Framing to Signal Risk

Humans are highly responsive to visual cues, especially color and iconography. Red triggers alertness and caution due to cultural and biological associations with danger. A trash icon or warning triangle adds visual weight and anchors the user's attention where it matters most.

[Image: Confirmation Dialogs: Use Visual Framing to Signal Risk]

6. Minimize Visual Noise

Excessive layout complexity results in eye strain and slower cognitive processing. A clean, centered design creates a single visual direction, reducing saccades (rapid eye movements) and allowing users to process the dialog as a whole — not in fragments.

[Image: Confirmation Dialogs: Minimize Visual Noise]

7. Introduce Friction for Critical Actions

When users are about to perform irreversible or high-risk actions, adding a moment of friction — like typing "DELETE," checking a confirmation box, or waiting 5 seconds — interrupts impulsive behavior and forces a mental pause.

This deliberate pause shifts the user from what psychologist **Daniel Kahneman, in Thinking, Fast and Slow**, calls System 1 thinking (fast, automatic, intuitive) to System 2 thinking — slower, more analytical, and better suited for evaluating consequences.

This brief disruption helps prevent costly errors, especially in high-stakes moments like deleting accounts, transferring large sums of money, or changing privacy settings.

[Image: Confirmation Dialogs: Introduce Friction for Critical Actions]

Accessibility Considerations

A well-designed confirmation dialog must be accessible to **all users** , including those who rely on assistive technologies like screen readers or keyboard navigation. Accessibility is not just a legal or ethical requirement — it is a fundamental part of creating **inclusive, respectful, and trustworthy** user experiences.

Here is how to ensure your confirmation dialog works for everyone:

1. Screen Reader Support. Use proper ARIA roles and labels to help screen readers announce the dialog clearly and in the correct order:
? role="alertdialog" tells assistive tech that this is an urgent, interruptive modal that should be read immediately.

NOTES

Further study materials Lesson 1

Transcript

? **aria-labelledby** connects the dialog to its title (e.g., "Delete this file permanently?").
? **aria-describedby** connects the dialog to its description (e.g., "This action cannot be undone.").
Together, these attributes give screen reader users the same understanding of the dialog as sighted users.

2. Keyboard Navigation. Make sure users can operate the dialog without a mouse:
? Pressing Tab moves between interactive elements (e.g., buttons).
? Enter activates the selected action.
? Escape should close the dialog (when safe to do so).
? The keyboard focus must remain inside the dialog until the user dismisses it.
Additionally, the default focus should land on the safe or non-destructive option (e.g., "Cancel"), not the destructive action — this helps prevent accidental confirmation.

3. Visual Contrast and Clarity. Users with low vision or color blindness should be able to read and understand your dialog without difficulty:
? Ensure high contrast between text and background.
? Do not rely on color alone to convey meaning — use icons, labels, and clear wording.
? Make button states (hover, focus, disabled) visually distinct.

The Future of Confirmation UX

These boxes may seem simple, but far more thought goes into them than most realize.

> *" In 2014, in the article Designing Confirmation by Andrew Coyle, Andrew Coyle wrote about how flat design made buttons less obvious, increasing the risk of mistakes. He argued for more deliberate, physical interactions. Today, companies like Amazon and Uber use swipe gestures for confirmation — requiring more conscious action.*

Looking ahead, interfaces are evolving: AI, VR/AR, wearables, voice and gesture input. How will confirmations work in these environments?

I think in the near future, confirmation patterns will become more invisible, adaptive, and intelligent — driven by context, behavior, and personalization. Instead of showing a dialog, the system might:

1. Predict risky behavior based on user patterns and gently intervene before errors occur.
2. Use ambient feedback — subtle haptics, voice prompts, or AR overlays — to guide decision-making.

NOTES

Further study materials Lesson 1

Transcript

3. Trigger confirmations only when anomalies are detected, such as unusual transaction amounts or unfamiliar locations.

As AI and sensor-rich environments advance, confirmations may evolve from explicit "Are you sure?" prompts into silent protectors — always active, rarely visible, and deeply human-aware.

A button that requires the user to press and hold for a few seconds before an irreversible or critical action (like deleting an account) is executed. This method, **"Hold to Confirm"** , adds intentional friction, prompting users to pause and consider their action — effectively reducing accidental taps. The holding gesture shifts the action from reflex to conscious decision-making. It is a lightweight yet powerful way to increase safety without cluttering the interface.

[Image: Hold to Confirm by www.scrollxui.dev]

Conclusion: Building Trust Through Friction

In this article, we explored:

- Why confirmation dialogs exist and how intentional friction helps prevent user error
- When confirmation is absolutely necessary — and when it becomes annoying noise
- Alternative patterns, like Undo or delayed actions, that reduce friction without compromising safety
- The full anatomy of a good confirmation dialog — from title and buttons to visual emphasis and psychological cues

> *" The key takeaway: Confirmation dialogues are not about clicks — they are about trust.*

A well-designed confirmation not only prevents mistakes but also makes a product safer, more predictable, and more respectful of the user's time and data. So next time you see "Delete?" or "Are you sure?", ask yourself: is this just another interruption, or a thoughtfully crafted safety net? And maybe you will start thinking about how you would design confirmations for the next generation of interfaces.

To bring this idea to life, I decided to experiment and design my own set of confirmation dialogs — applying all the principles covered in this article.

[Image: Confirmation Dialogs: Designed by Dmitry Sergushkin]

Even the smallest details — like the label on a cancel button, or a subtle animation — can make the difference between stress and confidence. Designing confirmation dialogs is not about adding barriers. It is about creating thoughtful pauses that respect the user's mind — not fight against it. Because at the end of the day,

NOTES

Further study materials Lesson 1

Transcript

good design is not about blocking users — it is about protecting them without ever making them feel powerless.

[Image: Confirmation Dialogs: Designed by Dmitry Sergushkin]

Thank you for reading, and please tell me — did you find this article helpful? ? I would love to hear your thoughts!

Here are two questions for you:

1. Have you ever ignored a confirmation dialog — and regretted it?

2. In your opinion, what is the most user-friendly confirmation pattern you have seen in a product?

Drop a note in the comments — I would love to discuss! ?

Follow me further for more UX insights ?

Say hello at ?: ux.sergushkin@gmail.com

Visit my Website 🌎: dmitrysergushkin.com

LinkedIn | Behance | Medium | Layers | Dribbble | Twitter

NOTES

Exercises • Lesson 1

EXERCISE 1 • True or false

CONTEXT

Flet's AlertDialog provides a way to prompt users for confirmation. It offers a modal property that controls how the dialog behaves when the user interacts outside of it.

QUESTION • Indicate whether the information is true or false

By default, clicking outside an AlertDialog in Flet closes the dialog.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 2 • Multiple choice

CONTEXT

In Flet, the AlertDialog can behave differently when a user clicks outside its boundaries. Understanding this behavior is crucial for choosing the right dialog mode for different scenarios.

QUESTION • Select the correct option

Which property of an AlertDialog controls whether clicking outside the dialog closes it?

- A) modal
- B) dismissible
- C) overlay
- D) open

NOTES

Exercises • Lesson 1

EXERCISE 3 • True or false

CONTEXT

When developing mobile apps in Flutter, developers often need to display alerts to users. Flutter provides two classes for creating alert dialogs: AlertDialog and CupertinoAlertDialog.

QUESTION • Indicate whether the information is true or false

The AlertDialog class in Flutter is specifically designed to provide a look and feel consistent with Android's Material Design.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 4 • Multiple choice

CONTEXT

In Flutter, AlertDialog and CupertinoAlertDialog are used to display alerts to users, with different aesthetic styles.

QUESTION • Select the correct option

Which of the following parameters is available in both AlertDialog and CupertinoAlertDialog?

A shape

B elevation

C actions

D backgroundColor

NOTES

Exercises • Lesson 1

EXERCISE 5 • True or false

CONTEXT

Confirmation dialogs are intentional interruptions that ask users to confirm an action. They are designed to prevent errors, but overusing them can lead to dialog fatigue.

QUESTION • Indicate whether the information is true or false

Confirmation dialogs should be used for all actions, including reversible ones.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 6 • Multiple choice

CONTEXT

The article discusses the appropriate use of confirmation dialogs in user interfaces, emphasizing that they should serve as intentional friction to prevent errors, but only in specific situations.

QUESTION • Select the correct option

According to the article, in which situation is a confirmation dialog most justified?

- A Only for financial transactions
- B For all user actions to ensure maximum safety
- C When the action is reversible and can be undone
- D When the action is irreversible and has serious consequences

NOTES

Exercise Answers

LESSON 1

EX.01 • True or false

True.

When modal is set to False, which is the default, clicking outside the dialog dismisses it. This is suitable for low-stakes prompts.

I got it right I got it wrong

EX.02 • Multiple choice

Option A — “modal”

The 'modal' property, when set to True, prevents the dialog from closing on outside clicks. When False (default), clicking outside closes the dialog. This directly controls the dismissible behavior.

I got it right I got it wrong

EX.03 • True or false

True.

AlertDialog implements the Material Design aesthetic for alerts, while CupertinoAlertDialog mimics the iOS style. This allows developers to choose the dialog style that matches the platform.

I got it right I got it wrong

EX.04 • Multiple choice

Option C — “actions”

The 'actions' parameter, which is a list of widgets for buttons, is common to both AlertDialog and CupertinoAlertDialog.

I got it right I got it wrong

EX.05 • True or false

False.

The article states that for reversible actions, confirmation is redundant; instead, an Undo pattern is more elegant and prevents dialog fatigue.

I got it right I got it wrong

EX.06 • Multiple choice

Option D — “When the action is irreversible and has serious consequences”

The article explicitly states that confirmation dialogs are justified for irreversible actions (e.g., permanent deletion) and actions with serious consequences (e.g., financial transactions). They protect users from costly mistakes.

I got it right I got it wrong

Chapter 05 • Dialogs, Overlays, and User Feedback

Lesson 02 • Progress Indicators and Loading States

27 pages • 36 min

elevify

Lesson 02 • Progress Indicators and Loading States

36 min • 27 pages

◎ What you will learn

Adds ProgressBar and ProgressRing to communicate async operation status. Proper loading states are essential for apps that fetch data or process files.

Progress Indicators and Loading States

Summary

Why and How to Use Progress Indicators

Progress indicators show users that something is happening during slow tasks, preventing confusion or repeated clicks. Flet offers determinate mode for known durations (e.g., file uploads) and indeterminate mode for unknown timing (e.g., server requests). The two controls are ProgressBar, a horizontal bar for page-level feedback, and ProgressRing, a compact circular spinner for buttons or overlays. Both support either mode, helping you set clear expectations and build trust.

Using ProgressBar and ProgressRing

To use ProgressBar, set value=None for indeterminate looping or a decimal 0–1 for exact progress (e.g., value=0.6 fills 60%). Style with color and bgcolor using hex codes or named colors. ProgressRing works similarly: no value keeps it spinning, a value draws a partial arc. stroke_width controls ring thickness, important for small containers. Both controls adapt to your layout and feedback needs.

Basic Show-Run-Hide Pattern

The pattern has four steps: start with the control hidden. On user action, set visible=True and call page.update(). Run your async task (await an API call or database query). After task completion, set visible=False and update again. The await frees the UI to show the spinner. Always update before and after the await to force Flet to redraw. This simple pattern prevents a silent UI.

Full-Screen Overlay and Loop Updates

For blocking tasks like loading initial data, use a full overlay. Build it with a Stack: put your main content first, then a semi-transparent Container with a centered ProgressRing. Toggle its visibility. When progress is known (e.g., processing items), update the bar's value inside a loop by dividing step by total for a fraction 0–1. Call page.update() each iteration for smooth filling. This gives detailed feedback.

Button Spinner and Safe Async

Replace a button's content with a small ProgressRing on click to prevent double-clicks and show immediate feedback. Disable the button, swap content, then restore after task. Always wrap async calls in a try-finally block: show spinner in try, await request, then hide in finally. This ensures the spinner disappears even on error. Use a matrix: overlay ProgressBar for known full-screen, overlay ProgressRing for unknown, inline ProgressBar for known tasks, small ProgressRing for unknown inline tasks.

NOTES

Progress Indicators and Loading States

Transcript

Adding Loading Feedback with Flet Progress Indicators

When your app fetches data or runs a slow task, users need to see that something's happening. Without feedback, they might click again, wait confused, or think your app's broken. This lesson fixes that.

You'll learn to use Flet's ProgressBar and ProgressRing to give users clear, professional loading feedback during async operations. A silent UI feels broken — progress indicators turn silence into a clear message.

Determinate vs Indeterminate Modes

Flet provides two distinct progress modes. Determinate indicates known duration, like a file upload where you can track exact bytes. Indeterminate loops for unknown timing, like an animated spinner waiting for server data.

Choosing the right mode shapes user expectations and trust. Determinate signals a measurable end; indeterminate signals an open-ended wait. Clarity is key.

Two Progress Controls in Flet

Flet gives you two controls. ProgressBar is a horizontal bar that stretches across a container — great for page-level progress or form feedback. ProgressRing is a compact circular spinner — it fits inside buttons, cards, or centered overlays.

Both support determinate and indeterminate modes. Pick based on your layout and the feedback you want to give.

Using ProgressBar in Code

When you set `value` to `None`, the bar enters indeterminate mode and loops continuously. That's useful when you don't know how long something will take. For exact progress, pass a decimal between 0 and 1 — `value=0.6` fills to 60%.

You can style the bar with `color` and `bgcolor`; use hex codes or named colors to match your app's theme perfectly.

Using ProgressRing in Code

ProgressRing works the same way. No value keeps it spinning indefinitely. A value between 0 and 1 draws a partial arc showing exactly how far the task is.

`stroke_width` controls the ring's thickness — important when placing it inside a small button or compact card. Keep the thickness proportional to the container size.

NOTES

Progress Indicators and Loading States

Transcript

The Basic Pattern: Show, Run, Hide

The pattern is simple: four steps. Start with the control hidden. When the user triggers an action, set `visible=True` and call `page.update()` — that forces Flet to redraw.

Then run your async task. Once it finishes, flip `visible` back to `False` and update again. Without that update call, the UI won't change.

Wiring an Async Function

You define an async function. Set the progress control to visible, then call `page.update()`. Next, `await` your actual task — an API call, database query, or any slow operation.

When it finishes, hide the progress control and call `page.update()` again. The `await` frees the UI thread to render the spinner while the background work completes. Always update before and after the await.

Full-Screen Overlay for Blocking Tasks

A small spinner isn't enough for tasks that block the whole screen, like loading initial page data. You need a full overlay covering the UI and preventing interaction with incomplete content.

In Flet, build this with a `Stack`: place your main content, then layer a semi-transparent `Container` with a centered `ProgressRing`. Toggle its visibility like any other control.

Building the Overlay in Code

The `Stack` holds two children: your main page content and the overlay. The overlay is a full-size `Container` with a semi-transparent black background and a centered `ProgressRing` inside.

Start with `visible=False`. When a blocking task begins, flip it to `True`. The overlay covers everything underneath, so users can't click buttons or interact with stale data while the task runs.

Updating Progress in a Loop

When you know the progress — say processing ten items one by one — update the bar's value inside the loop. Each iteration, divide the step by the total to get a fraction between 0 and 1.

Assign it to the bar and call `page.update()`. The bar fills smoothly as the loop runs. This gives users a far more informative experience than a spinning indeterminate bar.

Common Mistake: Forgetting page.update()

This is the most common mistake beginners make. You set `visible=True` or change a value, but nothing appears. The fix is

NOTES

Progress Indicators and Loading States

Transcript

simple: call `page.update()`. Flet batches UI changes until you force a redraw.

Get into the habit: after every control change, call `page.update()`.

Replacing Button Content with a Spinner

A polished pattern is to replace the button's content with a spinner while the action runs. When the user clicks Submit, you disable the button and swap its content for a small `ProgressRing`.

This prevents accidental double-clicks and gives immediate feedback at the point of interaction. When the task finishes, restore the original content and re-enable the button. A small detail that makes apps feel professional.

Safe Async with try-finally

When you hook up progress indicators to real httpx API calls, always wrap the call in a `try-finally` block. Inside `try`, show the spinner, `await` the request, then process the response.

Inside `finally`, hide the spinner and call `page.update()`. That `finally` block runs no matter what — success or exception — so your spinner can never get stuck on screen after an error.

Choosing the Right Control

Pick the right control with this matrix:

- Full-screen task, known progress: overlay `ProgressBar`
- Full-screen task, unknown duration: overlay `ProgressRing`
- Inline task, known progress: `ProgressBar` in layout
- Inline task, unknown duration: small `ProgressRing`

Your Complete Loading Toolkit

You now have the full toolkit for loading states in Flet. Use an inline `ProgressBar` when you know the task's progress. Block the screen with an overlay `ProgressRing` for blocking tasks.

Replace a button's content with a spinner for immediate feedback. Combine these with async functions and `try-finally`, and your apps handle every slow operation cleanly and professionally.

NOTES

Further study materials Lesson 2

Content type: Text

Designing Loading Indicators for Better User Experience and Engagement

Available at https://medium.com/@Flowmapp/what-type-of-loading-and-progress-indicators-implement-in-the-app-4efc0a1657d8

Full Content

Summary

Importance of Loading and Progress Indicators

Loading and progress indicators are important for positive interface perception. They give users a sense of control and show system status. When speed is low, they maintain interest until an action completes. According to Jakob Nielsen's heuristics, visibility of system status improves user experience. The choice of indicator depends on device power and internet speed. This article explains the differences and best practices.

Indeterminate vs Determinate Indicators

Indeterminate indicators are used when loading time depends on the user's device or internet speed, so the duration is unknown. Determinate indicators show exact progress in percentage and estimated time. Indeterminate indicators assure users the app is working, while determinate ones let users estimate wait time. The article includes animated examples for both types. Proper selection prevents user confusion and improves experience.

Best Practices: Single Indicator, Template, Progressive Loading

One best practice is to combine multiple downloads into a single loading indicator to avoid overwhelming users. Another is using a screen template instead of a blank screen to show upcoming content. Progressive loading loads essential content first, then secondary data. These methods reduce perceived wait time and keep users engaged. The article provides visual examples for each technique.

Best Practices: Placement, Uninterrupted Use, Estimated Time

Strategic placement of the indicator guides users: at bottom when swiping up, full page when swiping down. For long waits, use a small background window instead of full screen and allow cancel or pause. Showing estimated remaining time with a countdown or percentage counter reduces anxiety. These features improve user satisfaction and manage expectations during downloads.

NOTES

Further study materials Lesson 2

Summary

Best Practices: Progress Hints, Integrated Indicators, Entertainment

During long loads, show progress hints like animations to indicate the process is ongoing. Integrate the indicator into other UI elements for a smooth transition to a success icon. Entertainment elements like games keep users busy and reduce stress. These creative approaches enhance the overall app experience and user interest, as shown in the article's examples.

NOTES

Further study materials Lesson 2

Transcript

What Types of Loading and Progress Indicators Should You Implement in the App?

Indeterminate and determinate loading and progress indicators: what is the difference? Check the specific cases of their proper use.

Loading and progress indicators play an important role in the context of how positively the interface is perceived. Demonstrating download progress to users gives them a sense of control over the process of interacting with the application. Obviously, the system must have a speedy response to any action.

However, when it is impossible to provide the required speed, you need to organically beat the loading time to maintain interest in your application until the action is completed. After all, according to Jakob Nielsen's ten heuristics, visibility of the state of a system improves overall user experience and ease of interaction with the UI. Below, we will explain the differences between indeterminate and determinate indicators and highlight the best practices for their implementation.

What Is the Difference Between Indeterminate and Determinate Loading Indicators?

When creating a loading indicator, you should consider factors such as the power of user devices and the speed of their Internet connection. If they play a significant role in loading the specific element, this will lead you to a correct decision.

Indeterminate indicators

If the load of the particular element depends on the Internet speed and parameters of the end user's device, in that case, you should choose an indeterminate indicator. This way, users will be confident that the application is working and will not think about a malfunction or error.

[Image: Designed by YesYou® | Isaac Sanchez]

Determinate indicators

If your application can determine in percentage terms how much of the loading process is complete and how long it will take, then you should choose a determinate indicator. This will show users the level of progress and allow them to estimate the wait time.

[Image: Designed by Outcrowd]

9 Best Loading and Progress Indicators to Use

Let's look at examples of how you can organically use loading

NOTES

Further study materials Lesson 2

Transcript

indicators without compromising the user experience of your application.

Single loading indicator

If you need to carry out several simultaneous downloads on one application screen, combine them into one common one for all processes. This way, users will not be burdened with the feeling of long waits and overload. In the example below, you can see how such an indicator can be implemented.

[Image: Designed by Ash C]

Screen template

If you want to improve your app's user experience during loading, you can create a loading space template instead of a blank screen. Thus, users will get an understanding of what will happen next and will not be distracted from the tasks at hand.

[Image: Designed by Shane Doyle]

Progressive loading

One of your main goals in building a responsive and visually pleasing interface is to minimize the time it takes for content to load on the screen to maximize user retention. So, you can use the method of progressive page loading, where the most essential information will be loaded first, followed by the secondary data.

[Image: Designed by Gabriel Dzieslaw]

Strategic placement of loading indicators

Pay attention to the correct placement of the loading indicator on the screen. So, if the user swipes up, the indicator should appear at the bottom of the page, hinting that the content will be displayed there. The same goes for swiping down. In this case, the indicator should be depicted throughout the page, showing the user what will happen at the next step.

[Image: Designed by Yaroslav Abdulzade]

Uninterrupted use of the application

Long waits for loading make users feel uncomfortable. Thus, creating a small background loading window would be best instead of taking up the entire screen space. Also, valuable features will be the ability to stop and cancel downloads. By ensuring a smooth experience with your app, users will get greater satisfaction while interacting with it.

[Image: Designed by Alireza Kian]

Estimated remaining load time

To reduce user anxiety during loading, you can create a percentage

NOTES

Further study materials Lesson 2

Transcript

counter that shows progress. Another great feature is the countdown until the download is complete and the ability to stop or pause it. This way, you can improve user satisfaction and manage user expectations.

[Image: Designed by Aaron Iker]

Download progress hints

If the application needs to load for a long time, it is crucial for you to make it clear to the user that a lag or freeze has not occurred. To avoid disappointment, you must show that the process is going as planned. This way, you can create different animations or progress indicators that will keep users calm and improve the overall perception of your digital solution.

[Image: Designed by Cadabra Studio]

Loading indicator integrated into other elements

The idea of integrating the loading indicator into other UI elements of your app will give your users a more holistic experience. This solution also offers users a clearer understanding of the relationship between an action and its result. In addition, a smooth transition from the indeterminate indicator to the successful download icon will increase the overall impression of your product.

[Image: Designed by Shivam Kaushik]

Entertainment elements

You can use entertainment elements to reduce user stress during long loading times. Thanks to this, you can keep them busy with something interesting while completing the download process. This, in turn, will increase users' interest in your application and improve its perception compared to similar solutions.

[Image: Designed by JIANGGM]

We hope that our tips on implementing loading indicators will help you create digital solutions that will meet your audience's needs and wishes. However, this part of the work on UI is just a drop in the ocean of tasks assigned to the web designer from project to project. Therefore, you may need additional software tools to simplify your tasks. In particular, if you want to speed up and optimize the development of user flows, user journeys, and prototypes, you can try our comprehensive solution.

*[Image: https://miro.medium.com/1*_WtP4R_z6IljuVKjhBdpLA.jpeg]*

NOTES

Further study materials Lesson 2

Content type: Text

Flutter CircularProgressIndicator Modes Customization and Best Practices

Available at https://mailharshkhatri.medium.com/building-a-circular-progress-indicator-6594580e867d

Full Content

Summary

Introduction to CircularProgressIndicator

In modern mobile apps, showing progress during loading tasks is key for user experience. Flutter's CircularProgressIndicator widget provides a simple animated spinner. It tells users that an operation is ongoing, like fetching data or uploading files. The widget is customizable and easy to use. This article explores how to build and customize it, covering properties, use cases, and examples to create effective loading indicators.

Key Properties and Simple Usage

The CircularProgressIndicator has properties like value (for determinate progress), color, backgroundColor, strokeWidth, and semanticsLabel. With no value, it spins indefinitely. A basic example centers the indicator in a Scaffold. Default color uses the theme primary. This code shows a simple loading screen. The indicator is 36x36 pixels and works well in most layouts.

Customizing Appearance and Determinate Progress

Customize the indicator with color, backgroundColor, strokeWidth, and semanticsLabel for accessibility. For measurable tasks, set the value property (0.0 to 1.0) to create a determinate indicator. An example simulates a download by incrementing progress every 500ms. The arc fills proportionally, and a text displays the percentage. A button starts the simulation, resetting progress to 0.0.

Using in Dialogs and Animated Colors

Use the indicator in a dialog for async tasks like API calls. A loading dialog with a CircularProgressIndicator and text shows progress, and it auto-closes after a delay. For dynamic color, use valueColor with an AnimationController. An example animates the arc from red to blue over 3 seconds, repeating. This creates a visually engaging effect.

Best Practices, Pitfalls, and Conclusion

Pair the indicator with contextual text. Use theme colors for consistency. Add semanticsLabel for accessibility. Adjust size with SizedBox. Common pitfalls: indicator too small (fix with SizedBox or

NOTES

Further study materials Lesson 2

Summary

thicker stroke), no progress feedback (use determinate or text), color clashes (add backgroundColor), unresponsive UI (run tasks asynchronously). The CircularProgressIndicator enhances user experience. Experiment and follow best practices for polished apps.

NOTES

Further study materials Lesson 2

Transcript

*[Image: https://miro.medium.com/1*BikLcwKZ8sc7cTmZg8ZXVQ.jpeg]*

Building a Circular Progress Indicator

Introduction

In modern mobile applications, providing feedback during loading or processing tasks is crucial for a smooth and engaging user experience. In Flutter, Google's UI toolkit, the `CircularProgressIndicator` widget offers a simple yet effective way to visually indicate ongoing operations, such as fetching data, uploading files, or processing user inputs. With its customizable appearance and seamless integration into various layouts, this widget helps keep users informed and engaged. This article explores how to build and customize a `CircularProgressIndicator` in Flutter, diving into its properties, use cases, and practical examples to help you create intuitive and visually appealing loading indicators.

What is a CircularProgressIndicator?

The `CircularProgressIndicator` widget in Flutter is a Material Design component that displays a circular, animated loading indicator. It consists of a spinning arc that rotates to signify that an operation is in progress. The widget is highly customizable, allowing developers to adjust its size, color, thickness, and animation behavior. It's commonly used in scenarios like network requests, file downloads, or form submissions to reassure users that the app is actively working.

Why Use a CircularProgressIndicator?

- **User Feedback**: Informs users that a task is ongoing, reducing uncertainty.
- **Visual Clarity**: Provides a clear, animated indication of progress.
- **Customizability**: Supports adjustments to color, size, and stroke width for branding.
- **Ease of Use**: Integrates effortlessly into any Flutter layout.
- **Consistency**: Aligns with Material Design for a professional and familiar look.

The `CircularProgressIndicator` is a versatile tool for enhancing the user experience during asynchronous operations.

Key Properties of CircularProgressIndicator

The `CircularProgressIndicator` widget offers several properties to tailor its appearance and behavior:

- **value**: A `double` (0.0 to 1.0) indicating progress for determinate indicators; `null` for indeterminate (continuous spinning).

NOTES

Further study materials Lesson 2

Transcript

- **color**: The color of the spinning arc, defaulting to the theme's primary color.
- **backgroundColor**: The color of the circular track behind the arc.
- **strokeWidth**: The thickness of the arc (default: 4.0).
- **semanticsLabel**: An accessibility label for screen readers.
- **valueColor**: An `Animation<Color>` for dynamic color changes during animation.

These properties allow precise control over the indicator's look and functionality.

Building a Circular Progress Indicator

Let's explore how to implement and customize the `CircularProgressIndicator`, starting with basic usage and progressing to advanced scenarios with dynamic progress and custom animations.

1. Basic Indeterminate CircularProgressIndicator

The simplest use case is an indeterminate indicator that spins continuously to show an ongoing task.

Example: A basic loading indicator:

```
import 'package:flutter/material.dart';
void main() {
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({Key? key}) : super(key: key);
@override
Widget build(BuildContext context) {
return MaterialApp(
theme: ThemeData(primarySwatch: Colors.blue),
home: const BasicProgressScreen(),
);
}
}
class BasicProgressScreen extends StatelessWidget {
const BasicProgressScreen({Key? key}) : super(key: key);
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Basic
CircularProgressIndicator')),
body: const Center(
child: CircularProgressIndicator(),
),
);
}
}
```

In this example:

NOTES

Further study materials Lesson 2

Transcript

- The `CircularProgressIndicator` is centered in the `Scaffold`'s body.
- With no `value` specified, it runs as an indeterminate indicator, spinning continuously.
- The default color is the theme's primary color (blue, from `primarySwatch`).
- The indicator's size is approximately 36x36 pixels, fitting most layouts.

2. Customizing Appearance

Customize the indicator's color, size, and stroke width to match your app's design.

Example: A styled indeterminate indicator:

```
class StyledProgressScreen extends StatelessWidget {
const StyledProgressScreen({Key? key}) : super(key: key);
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Styled
CircularProgressIndicator')),
body: Center(
child: Column(
mainAxisAlignment: MainAxisAlignment.center,
children: const [
CircularProgressIndicator(
color: Colors.teal,
backgroundColor: Colors.grey,
strokeWidth: 6.0,
semanticsLabel: 'Loading content',
),
SizedBox(height: 16),
Text('Loading...', style: TextStyle(fontSize: 18)),
],
),
),
);
}
}
```

In this example:

- `color: Colors.teal` sets the spinning arc to teal.
- `backgroundColor: Colors.grey` adds a grey circular track for contrast.
- `strokeWidth: 6.0` increases the arc's thickness for a bolder look.
- `semanticsLabel: 'Loading content'` improves accessibility for screen readers.
- A `Text` widget below the indicator provides additional context.

3. Determinate CircularProgressIndicator

NOTES

Further study materials Lesson 2

Transcript

For tasks with measurable progress (e.g., file downloads), use a determinate indicator by setting the `value` property.

Example: A progress indicator with simulated download:

```
class DeterminateProgressScreen extends StatefulWidget {
const DeterminateProgressScreen({Key? key}) : super(key:
key);
@override
_DeterminateProgressScreenState createState() =>
_DeterminateProgressScreenState();
}
class _DeterminateProgressScreenState extends
State<DeterminateProgressScreen> {
double _progress = 0.0;
void _simulateDownload() {
setState(() {
_progress = 0.0;
});
Future<void>.delayed(const Duration(milliseconds: 500),
() {
if (_progress < 1.0 && mounted) {
setState(() {
_progress += 0.1;
});
_simulateDownload();
}
});
}
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Determinate
CircularProgressIndicator')),
body: Center(
child: Column(
mainAxisAlignment: MainAxisAlignment.center,
children: [
CircularProgressIndicator(
value: _progress,
color: Colors.purple,
backgroundColor: Colors.grey[200],
strokeWidth: 8.0,
),
const SizedBox(height: 16),
Text(
'Download: ${(_progress * 100).toStringAsFixed(0)}%',
style: const TextStyle(fontSize: 18),
),
const SizedBox(height: 16),
ElevatedButton(
onPressed: _simulateDownload,
child: const Text('Start Download'),
),
],
),
```

NOTES

Further study materials Lesson 2

Transcript

```
),
);
}
}
```

In this example:

- A `StatefulWidget` tracks `_progress` (0.0 to 1.0).
- `_simulateDownload` increments `_progress` every 500ms to mimic a download.
- `value: _progress` makes the indicator determinate, filling the arc proportionally.
- The `Text` widget displays the progress percentage.
- An `ElevatedButton` triggers the simulation, resetting `_progress` to 0.0.

4. CircularProgressIndicator in a Dialog

Use the indicator in a dialog to inform users during asynchronous operations, like API calls.

Example: A loading dialog with an indicator:

```
class DialogProgressScreen extends StatelessWidget {
const DialogProgressScreen({Key? key}) : super(key: key);
void _showLoadingDialog(BuildContext context) {
showDialog(
context: context,
barrierDismissible: false,
builder: (context) => AlertDialog(
content: Row(
children: const [
CircularProgressIndicator(color: Colors.blue),
SizedBox(width: 16),
Text('Fetching Data...'),
],
),
),
);
Future<void>.delayed(const Duration(seconds: 3), () {
Navigator.pop(context);
ScaffoldMessenger.of(context).showSnackBar(
const SnackBar(content: Text('Data Fetched')),
);
});
}
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Progress in Dialog')),
body: Center(
child: ElevatedButton(
onPressed: () => _showLoadingDialog(context),
child: const Text('Fetch Data'),
),
```

NOTES

Further study materials Lesson 2

Transcript

```
),
);
}
}
```

In this example:

- showDialog displays an `AlertDialog` with a `CircularProgressIndicator` and text.
- `barrierDismissible: false` prevents dismissing the dialog until the task completes.
- A 3-second delay simulates an API call, closing the dialog and showing a `SnackBar`.
- The indicator's blue color aligns with the app's theme.

5. Custom Animated CircularProgressIndicator

Create a custom indicator with dynamic color animation using `valueColor`.

Example: An indicator with animated color changes:

```
class AnimatedProgressScreen extends StatefulWidget {
const AnimatedProgressScreen({Key? key}) : super(key:
key);
@override
_AnimatedProgressScreenState createState() =>
_AnimatedProgressScreenState();
}
class _AnimatedProgressScreenState extends
State<AnimatedProgressScreen>
with SingleTickerProviderStateMixin {
late AnimationController _controller;
late Animation<Color?> _colorAnimation;
@override
void initState() {
super.initState();
_controller = AnimationController(
duration: const Duration(seconds: 3),
vsync: this,
)..repeat();
_colorAnimation = ColorTween(
begin: Colors.red,
end: Colors.blue,
).animate(_controller);
}
@override
void dispose() {
_controller.dispose();
super.dispose();
}
@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Animated
CircularProgressIndicator')),
```

NOTES

Further study materials Lesson 2

Transcript

```
body: Center(
child: Column(
mainAxisAlignment: MainAxisAlignment.center,
children: [
AnimatedBuilder(
animation: _colorAnimation,
builder: (context, child) => CircularProgressIndicator(
valueColor:
AlwaysStoppedAnimation(_colorAnimation.value),
backgroundColor: Colors.grey[200],
strokeWidth: 10.0,
),
),
const SizedBox(height: 16),
const Text('Loading with Color Animation', style:
TextStyle(fontSize: 18)),
],
),
),
);
}
}
```

In this example:

- A `StatefulWidget` with `SingleTickerProviderStateMixin` manages an `AnimationController`.
- `ColorTween` animates the arc's color from red to blue over 3 seconds, repeating.
- `AnimatedBuilder` rebuilds the `CircularProgressIndicator` with the current color from `_colorAnimation`.
- `valueColor` applies the dynamic color, creating a visually engaging effect.
- The controller is disposed to prevent memory leaks.

Best Practices for CircularProgressIndicator

1. Provide Context:

- Pair the indicator with text (e.g., "Loading...") to clarify the task:

```
Column(children: [CircularProgressIndicator(),
Text('Loading...')])
```

2. Use Theme Colors:

- Align the `color` with the app's theme for consistency:

```
color: Theme.of(context).primaryColor
```

3. Enhance Accessibility:

- Add `semanticsLabel` for screen readers:

```
semanticsLabel: 'Loading data'
```

NOTES

Further study materials Lesson 2

Transcript

4. Optimize Size and Thickness:

• Adjust `strokeWidth` and wrap in `SizedBox` for appropriate sizing:

```
SizedBox(width: 50, height: 50, child:
CircularProgressIndicator())
```

5. Test Across Scenarios:

• Verify appearance in dialogs, full-screen layouts, and with different network speeds.

Common Pitfalls and Solutions

1. Indicator Too Small:

• **Problem**: The indicator is barely visible.

• **Solution**: Use `SizedBox` or increase `strokeWidth`:

```
SizedBox(width: 60, height: 60, child:
CircularProgressIndicator(strokeWidth: 6))
```

2. No Progress Feedback:

• **Problem**: Users don't know the task's status.

• **Solution**: Use `value` for determinate progress or add descriptive text.

3. Color Clashes:

• **Problem**: Indicator color blends with the background.

• **Solution**: Use `backgroundColor` for contrast:

```
backgroundColor: Colors.grey[200]
```

4. Unresponsive UI:

• **Problem**: Indicator doesn't animate during heavy tasks.

• **Solution**: Run tasks asynchronously with `Future` or `async/await`.

Conclusion

Building a `CircularProgressIndicator` in Flutter is a straightforward yet powerful way to provide feedback during asynchronous operations, enhancing the user experience with clear and engaging visuals. With its customizable properties for color, size, stroke width, and progress value, the widget fits seamlessly into various layouts, from full-screen loaders to dialogs. By incorporating advanced techniques like color animations or determinate progress, you can create indicators that align with your app's design and functionality.

Experiment with the examples provided, test your indicators across devices and scenarios, and follow best practices to ensure a polished user experience. With the

NOTES

Further study materials Lesson 2

Transcript

`CircularProgressIndicator` in your Flutter toolkit, you're ready to build professional-grade apps that keep users informed and engaged during every task!

NOTES

Exercises • Lesson 2

EXERCISE 1 • True or false

CONTEXT

Flet provides progress indicators for loading feedback. Two controls are available: ProgressBar and ProgressRing.

QUESTION • Indicate whether the information is true or false

Indeterminate mode is used for tasks with a known exact duration.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 2 • Multiple choice

CONTEXT

When adding progress indicators to Flet apps, developers often encounter a specific issue that prevents the UI from updating.

QUESTION • Select the correct option

What is the most common mistake when using progress indicators in Flet?

A. Not using a Stack for overlays.

B. Forgetting to call page.update() after changing the progress control's properties.

C. Setting the progress value to a number greater than 1.

D. Using the wrong progress control type.

NOTES

Exercises • Lesson 2

EXERCISE 3 • True or false

CONTEXT

Loading indicators can be categorized as determinate or indeterminate based on different factors.

QUESTION • Indicate whether the information is true or false

Determinate indicators should be used when the loading time depends on the user's internet speed and device parameters.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 4 • Multiple choice

CONTEXT

Loading indicators are important for user experience. There are two main types: indeterminate and determinate, each used in different scenarios.

QUESTION • Select the correct option

Which type of loading indicator should be used when the application can determine the exact progress percentage?

A) Single loading indicator

B) Indeterminate indicator

C) Progressive loading

D) Determinate indicator

NOTES

Exercises • Lesson 2

EXERCISE 5 • True or false

CONTEXT

When building Flutter apps, developers often need to indicate ongoing operations to users. The CircularProgressIndicator widget provides visual feedback during such tasks. It is important to understand its design origin and properties to use it effectively.

QUESTION • Indicate whether the information is true or false

The CircularProgressIndicator is a Cupertino design component.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 6 • Multiple choice

CONTEXT

In Flutter, the CircularProgressIndicator can be used to show progress for ongoing tasks. One key aspect is whether the progress is known (determinate) or unknown (indeterminate).

QUESTION • Select the correct option

How can you make the CircularProgressIndicator show determinate progress (a specific percentage)?

A. By setting the value property to a double between 0.0 and 1.0.

B. By setting the value property to null.

C. By using the backgroundColor property.

D. By setting the color property to a specific color.

NOTES

Exercise Answers

LESSON 2

EX.01 • True or false

False.

Indeterminate mode is used when the task duration is unknown; it loops continuously. Determinate mode is used when the exact duration is known, with a value between 0 and 1 showing progress.

I got it right I got it wrong

EX.02 • Multiple choice

Option B — "Forgetting to call page.update() after changing the progress control's properties."

Flet batches UI changes; without page.update(), the changes are not rendered on screen, so the progress indicator never appears or updates.

I got it right I got it wrong

EX.03 • True or false

False.

The lesson states that when loading depends on internet speed and device parameters, an indeterminate indicator is appropriate. Determinate indicators are used when the app can determine the exact progress and time remaining.

I got it right I got it wrong

EX.04 • Multiple choice

Option D — "Determinate indicator"

A determinate indicator shows the exact progress in percentage or time remaining, which is appropriate when the system can calculate the loading time.

I got it right I got it wrong

EX.05 • True or false

False.

The lesson explicitly states that CircularProgressIndicator is a Material Design component, not a Cupertino one. Cupertino design is used for iOS-style widgets in Flutter.

I got it right I got it wrong

EX.06 • Multiple choice

Option A — "By setting the value property to a double between 0.0 and 1.0."

The value property controls how much of the arc is filled; when set, the indicator shows determinate progress.

I got it right I got it wrong

Chapter 05 • Dialogs, Overlays, and User Feedback

Lesson 03 • BottomSheet and Drawer Controls

17 pages • 18 min

elevify

Lesson 03 • BottomSheet and Drawer Controls

18 min • 17 pages

◎ What you will learn

Implements BottomSheet for contextual actions and Drawer for side navigation. Both controls are common in mobile-first Flet applications.

BottomSheet and Drawer Controls

Summary

Introduction to Overlays in Flet

Overlays like BottomSheet and Drawer keep users on the same screen while offering contextual actions or navigation. BottomSheet slides up for actions tied to on-screen content, while Drawer slides in from the side for app-wide navigation. Choosing the right overlay reduces friction and makes the app feel effortless.

BottomSheet: Open, Configure, Close

To open a BottomSheet, create a BottomSheet control, populate it with content (e.g., Column with Text and buttons), then call page.show_bottom_sheet(). The sheet slides up and the screen dims. Use modal mode for decisions or persistent mode for supplementary info. Close by setting open=False and calling page.update(). Store a reference to the control to close it.

Drawer: Set Up and Navigate

Attach a Drawer to page.drawer and open it with page.open_drawer(). It slides from the left with a header and ListTile items. Each ListTile leads to a route; on_click navigates and then closes the drawer with page.close_drawer(). Always close the drawer after navigation to avoid blocking the new content.

When to Use BottomSheet vs Drawer

Use BottomSheet for actions tied to a specific item, like edit or share. Use Drawer for moving between major app sections, triggered by the app bar. A common pattern is a list with trailing icons: tapping the icon opens a BottomSheet with options for that item. Dismissal is similar for both.

Common Mistakes and Five-Step Flow

Common mistakes: forgetting page.update(); not closing Drawer after navigation; using BottomSheet for app navigation; nesting scrollable content without is_scroll_controlled=True. The five-step flow: define control, attach to page, trigger open, let user interact, close programmatically. BottomSheet uses show_bottom_sheet and open=False; Drawer uses page.drawer, open_drawer, and close_drawer.

NOTES

BottomSheet and Drawer Controls

Transcript

Introduction to Overlays

In this lesson you'll get to know two overlay controls in Flet: the BottomSheet and the Drawer. These are the patterns your users rely on for contextual actions or quick navigation without leaving the screen. By the end, you'll know how to open, configure, and close both controls programmatically with confidence.

Every time you push someone to a new screen just to show a few options, you add friction. Overlays fix that. A BottomSheet slides up from the bottom for contextual actions. A Drawer slides in from the side for navigation. Both keep users anchored to the current screen while giving them exactly what they need.

Two controls, two distinct jobs. The BottomSheet handles contextual actions — like sharing, editing, or deleting an item — tied directly to what the user sees. It keeps users on the current screen. The Drawer handles app-wide navigation, a menu for jumping between major sections. It moves users between them. Knowing which one to reach for is your first decision. Choose wrong, you add friction; choose right, it feels effortless.

BottomSheet

Opening a BottomSheet in Flet is straightforward. Create a BottomSheet control, populate it with whatever content you need—a Column of Text and buttons, for example—then call `page.show_bottom_sheet()`. Flet handles the animation and overlay automatically. The sheet slides up from the bottom, and the rest of the screen dims. That dimming tells the user this is a focused interaction.

This is the minimal pattern. You write a function that builds the BottomSheet, sets its `open` property to `True`, then calls `page.show_bottom_sheet`. The content is just a Container wrapping a Column—simple. You can drop any Flet control inside: buttons, a form, an image. Whatever you need. The core stays the same: build it, open it, pass it to the page.

Flet supports both modes. A modal BottomSheet dims the background and locks the rest of the UI until dismissed. Use it when you need a decision before moving on. A persistent sheet stays on screen alongside the main content without blocking it. Use it for supplementary info or tools the user can reference while still interacting with the page. The `is_scroll_controlled` property and the modal flag control this behavior.

Closing the sheet takes the same care as opening it. First, store a reference to your BottomSheet variable. Then, inside your close handler, set its `open` property to `False`. Finally, call `page.update()` to make the change take effect. Flet animates the sheet sliding back down automatically. This core pattern—store the

NOTES

BottomSheet and Drawer Controls

Transcript

control, change a property, refresh—works for every overlay in Flet.

Drawer

The Drawer slides in from the left edge of the screen. It's the default standard pattern for app-level navigation. At the top you place a DrawerHeader with branding or user info, and below it you add ListTile controls for each navigation destination. Flet makes this structure explicit and easy to compose, with just a few controls.

Actually, attaching a Drawer is simple: assign it to `page.drawer`. Then to open it, call `page.open_drawer()`. The standard trigger is an icon button in the AppBar — Flet can add that automatically when a drawer is present. Each ListTile in the drawer needs an `on_click` handler. Inside that handler, you handle navigation — for instance, swapping the page body — and then close the drawer.

ListTile is the basic building block of every Drawer. The `leading` parameter takes an Icon to identify each item visually. Setting `selected` to True highlights the active route — this keeps the user oriented. Inside `on_click`, you do two things: navigate to the target view, then close the drawer by calling `page.close_drawer`. Always close after navigation so the user lands on a clean screen.

The drawer close sequence is a three-step cascade. First, user taps a ListTile — `on_click` fires. Then you update the page content or route. Finally, call `page.close_drawer()`. That last call is easy to miss but critical. Skip it and the drawer stays on top of the new content, ruining the experience. Always close the drawer as the final step in your navigation handler.

Usage Comparison and Patterns

Keep this table handy. When a tap on a specific item opens an overlay tied to that item, reach for a BottomSheet. When you're moving between major app sections and the trigger is the app bar, use a Drawer. Dismissal works the same for both — tap outside or press close — but scope and trigger decide which one you need.

Here's a pattern you'll reach for all the time. A list of items, each with a trailing icon button. Tap the icon, and a BottomSheet pops up with actions for that specific item: edit, share, delete. Each action does its work, then closes the sheet. That's the action sheet pattern — one of the most common uses of BottomSheet in mobile and desktop apps.

Common Mistakes

Four common mistakes. First, forgetting `page.update()` after setting `open` to `False` — overlay stays. Second, not closing the Drawer after navigation. Third, using BottomSheet for app navigation — that's the Drawer's job. Fourth, nesting scrollable content inside a BottomSheet without

NOTES

BottomSheet and Drawer Controls

Transcript

`is_scroll_controlled=True` — sheet won't expand. Avoid these and overlays work correctly.

The Five-Step Flow

Every overlay in Flet follows this five-step flow. First, define the control. Next, attach it to the page. Then trigger the open. Let the user interact. Finally, close it programmatically. Whether you're working with a BottomSheet or a Drawer, this mental model applies. Internalize the flow, and adding any overlay to a Flet app becomes a predictable, repeatable task.

Key Takeaways

BottomSheet opens with `page.show_bottom_sheet()`. Close it by setting `open=False` and calling `page.update()`. Drawer attaches to `page.drawer`. Open with `page.open_drawer()`, close with `page.close_drawer()` after navigation. Match the control to the job—BottomSheet for contextual item actions, Drawer for app-level navigation. That’s the core takeaway.

NOTES

Further study materials Lesson 3

Content type: Text

Flutter Bottom Sheets: Modal, Persistent, Draggable, and Best Practices

Available at https://medium.com/fludev/flutter-bottomsheet-enhancing-user-interaction-10cf7e92ac8d

Full Content

Summary

Introduction to BottomSheet

In Flutter, a BottomSheet is a UI component that slides up from the bottom of the screen. It is used to show extra information or actions without changing the screen. Bottom sheets are good for menus, lists, or options while keeping users on the same page. There are two types: modal and persistent. Modal bottom sheets block other parts of the screen until closed. Persistent bottom sheets stay visible and allow interaction with the rest of the screen.

Modal Bottom Sheet

A modal bottom sheet covers part of the screen and needs user action to close. It is often used for tasks that need input from the user. The code example shows a simple modal bottom sheet with a title, text, and a close button. To show it, call showModalBottomSheet with a builder that returns the sheet's content. The sheet appears when the user taps a button on the home screen. This type is temporary and must be dismissed explicitly.

Persistent Bottom Sheet

A persistent bottom sheet stays on the screen even when the user interacts with other parts. It is great for toolbars or secondary actions. The example uses ScaffoldMessenger.showBottomSheet to create it. The sheet has a blue background, a title, text, and a close button. It remains visible until the user closes it. Unlike modal, it does not block the background, so users can tap other widgets while the sheet is open.

Customization and DraggableSheet

Bottom sheets can be customized to match your app's design. For example, a modal bottom sheet can have rounded corners and include list tiles for share, copy, and edit actions. For more dynamic content, use DraggableScrollableSheet. This allows users to resize the sheet by dragging. The example shows a list of 30 items that can be scrolled, with initial, minimum, and maximum sizes set. This is useful for larger content like forms or lists.

NOTES

Further study materials Lesson 3

Summary

Best Practices and Conclusion

Best practices: Use modal bottom sheets for short actions, persistent sheets for tools, and draggable sheets for large content. Test on different screen sizes. The BottomSheet widget is powerful for modern UIs. Experiment with examples to create interactive designs. The article encourages readers to try these ideas and share comments. It also invites support via Buy Me a Coffee. Overall, bottom sheets enhance user interaction without navigating away from the current page.

NOTES

Further study materials Lesson 3

Transcript

Flutter BottomSheet: Enhancing User Interaction

*[Image: https://miro.medium.com/1*zLmuBLKA4et28UQTOsQAoA.png]*

In Flutter, a **BottomSheet** is a versatile UI component that slides up from the bottom of the screen. It is widely used for displaying additional information or actions without navigating to a new screen. Bottom sheets are particularly effective for presenting content like menus, lists, or contextual options while maintaining a connection to the underlying page.

In this guide, we'll dive deep into **BottomSheet**, its types, usage, and practical examples to help you implement it effectively in your Flutter projects.

Types of Bottom Sheets

Flutter provides two main types of bottom sheets:

Modal Bottom Sheet

- Blocks interaction with the rest of the screen until dismissed.
- Commonly used for actions requiring user input.

Persistent Bottom Sheet

- Stays visible on the screen while still allowing interaction with other parts of the UI.
- Typically used for app-specific toolbars or contextual actions.

Modal Bottom Sheet

A **Modal Bottom Sheet** covers part of the screen and requires user action to dismiss it.

Example: Simple Modal Bottom Sheet

Code:

```
import 'package:flutter/material.dart';

void main() {
runApp(const MyApp());
}

class MyApp extends StatelessWidget {
const MyApp({super.key});

@override
Widget build(BuildContext context) {
```

NOTES

Further study materials Lesson 3

Transcript

```
return const MaterialApp(
home: HomeScreen(),
);
}
}

class HomeScreen extends StatelessWidget {
const HomeScreen({super.key});

void _showModalBottomSheet(BuildContext context) {
showModalBottomSheet(
context: context,
builder: (BuildContext context) {
return Container(
height: 200,
padding: const EdgeInsets.all(16.0),
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
const Text('Modal Bottom Sheet',
style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
const SizedBox(height: 10),
const Text('This is an example of a modal bottom
sheet.'),
ElevatedButton(
onPressed: () => Navigator.pop(context),
child: const Text('Close'),
),
],
),
);
},
);
}

@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('BottomSheet Example')),
body: Center(
child: ElevatedButton(
onPressed: () => _showModalBottomSheet(context),
child: const Text('Show Modal Bottom Sheet'),
),
),
);
}
}
```

Persistent Bottom Sheet

A **Persistent Bottom Sheet** remains visible even when the user interacts with other parts of the screen.

NOTES

Further study materials Lesson 3

Transcript

Example: Simple Persistent Bottom Sheet

Code:

```
class HomeScreen extends StatelessWidget {
const HomeScreen({super.key});

void _showPersistentBottomSheet(BuildContext context) {
ScaffoldMessenger.of(context).showBottomSheet(
(BuildContext context) {
return Container(
color: Colors.blue[100],
height: 200,
padding: const EdgeInsets.all(16.0),
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
const Text('Persistent Bottom Sheet',
style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
const SizedBox(height: 10),
const Text('This bottom sheet will remain visible until
dismissed.'),
ElevatedButton(
onPressed: () => Navigator.pop(context),
child: const Text('Close'),
),
],
),
);
},
);
}

@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('BottomSheet Example')),
body: Center(
child: ElevatedButton(
onPressed: () => _showPersistentBottomSheet(context),
child: const Text('Show Persistent Bottom Sheet'),
),
),
);
}
}
```

Customizing Bottom Sheets

Bottom sheets can be styled and customized to match your app's design.

NOTES

Further study materials Lesson 3

Transcript

Example: Customizing a Modal Bottom Sheet

Code:

```
void _showCustomBottomSheet(BuildContext context) {
showModalBottomSheet(
context: context,
shape: const RoundedRectangleBorder(
borderRadius: BorderRadius.vertical(top:
Radius.circular(20)),
),
builder: (BuildContext context) {
return Container(
height: 250,
padding: const EdgeInsets.all(16.0),
child: Column(
children: [
const Text('Custom Styled Bottom Sheet',
style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
const Divider(),
ListTile(
leading: const Icon(Icons.share),
title: const Text('Share'),
onTap: () {},
),
ListTile(
leading: const Icon(Icons.copy),
title: const Text('Copy Link'),
onTap: () {},
),
ListTile(
leading: const Icon(Icons.edit),
title: const Text('Edit'),
onTap: () {},
),
],
),
);
},
);
}
```

Advanced Use: DraggableScrollableSheet

For more dynamic content, use the **DraggableScrollableSheet**, which allows users to resize the bottom sheet.

Example: DraggableScrollableSheet

Code:

```
class DraggableBottomSheetExample extends StatelessWidget
{
```

NOTES

Further study materials Lesson 3

Transcript

```
const DraggableBottomSheetExample({super.key});

@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('Draggable Bottom
Sheet')),
body: DraggableScrollableSheet(
initialChildSize: 0.3,
minChildSize: 0.2,
maxChildSize: 0.8,
builder: (BuildContext context, ScrollController
scrollController) {
return Container(
color: Colors.blue[100],
child: ListView.builder(
controller: scrollController,
itemCount: 30,
itemBuilder: (context, index) {
return ListTile(title: Text('Item $index'));
},
),
);
},
),
);
}
}
```

Best Practices

1. **Use Modal Bottom Sheets for Temporary Actions**: Ideal for short interactions or selections.
2. **Persistent Bottom Sheets for Frequently Used Tools**: Great for toolbars or secondary navigation.
3. **DraggableScrollableSheet for Dynamic Content**: Suitable for larger, scrollable content like forms or lists.
4. **Test Responsiveness**: Ensure bottom sheets render well on different screen sizes and orientations.

Conclusion

The **BottomSheet** widget in Flutter is a versatile and powerful tool for creating modern, interactive UIs. Whether you need a modal, persistent, or draggable sheet, Flutter has you covered. Experiment with the examples above, and let your creativity shine!

Have questions or ideas for bottom sheets in your app? Drop a comment below and let's discuss! Don't forget to clap and follow for more Flutter insights!

If you found this story helpful, you can support me at Buy Me a

NOTES

Further study materials Lesson 3

Transcript

Coffee!

NOTES

Exercises • Lesson 3

EXERCISE 1 • True or false

CONTEXT

Flet provides overlay controls like BottomSheet and Drawer to present additional options or navigation without leaving the current screen.

QUESTION • Indicate whether the information is true or false

The Drawer is intended for app-level navigation between major sections.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 2 • Multiple choice

CONTEXT

In Flet, overlays like BottomSheet and Drawer are used to keep users on the current screen while providing additional actions or navigation.

QUESTION • Select the correct option

Which method is used to open a BottomSheet?

A page.open_drawer()

B page.add()

C page.open_bottom_sheet()

D page.show_bottom_sheet()

NOTES

Exercises • Lesson 3

EXERCISE 3 • True or false

CONTEXT

In Flutter, BottomSheet is a UI component that slides up from the bottom. It can be used for menus, actions, or additional information.

QUESTION • Indicate whether the information is true or false

Flutter provides three main types of bottom sheets: Modal, Persistent, and Draggable.

◯ TRUE ◯ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 4 • Multiple choice

CONTEXT

In Flutter, bottom sheets are used to display additional information or actions without navigating to a new screen. There are two main types: modal and persistent bottom sheets.

QUESTION • Select the correct option

What happens when a Modal Bottom Sheet is displayed?

A. It is typically used for app-specific toolbars or contextual actions.

B. It remains visible while allowing interaction with other UI.

C. It blocks interaction with the rest of the screen until dismissed.

D. It can be resized by the user using a DraggableScrollableSheet.

NOTES

Exercise Answers

LESSON 3

EX.01 • True or false

True.

According to the lesson, the Drawer handles app-wide navigation, such as jumping between major sections of the app, making this statement true.

I got it right I got it wrong

EX.02 • Multiple choice

Option D — "page.show_bottom_sheet()"

The lesson explicitly states that to open a BottomSheet, you call page.show_bottom_sheet(). Flet handles the animation and overlay automatically.

I got it right I got it wrong

EX.03 • True or false

False.

The lesson states that Flutter has two main types: Modal and Persistent. DraggableScrollableSheet is an advanced widget for dynamic content, not a separate type.

I got it right I got it wrong

EX.04 • Multiple choice

Option C — "It blocks interaction with the rest of the screen until dismissed."

The lesson states that a Modal Bottom Sheet covers part of the screen and requires user action to dismiss, blocking interaction with other UI elements.

I got it right I got it wrong

Chapter 05 • Dialogs, Overlays, and User Feedback

Practical project of the chapter

15 pages

elevify

Practical project of the chapter

15 pages

Dialogs, Overlays, and User Feedback

Driving Question

Build a Flet app with a confirmation dialog, loading overlay, and snackbar, and record a short video demonstrating each feedback mechanism in action.

Production Format

Argumentative essay — 800 to 1500 words.

Context and Provocation

Objective

Produce a Flet application that implements a confirmation dialog for deleting a journal entry, a loading overlay for a simulated data save, and a snackbar for positive feedback. This project exercises AlertDialog, ProgressRing with overlay, and SnackBar, building components that will later integrate into the full journal app.

Creative Brief

Your friend loves the journal app you've been building together, but now they're worried: "What if I accidentally delete an entry? And sometimes the app feels like it's thinking without letting me know—can you add a spinner or something? Oh, and when I do something right, it should give me a little thumbs-up!" They want the app to feel safe and responsive. Your mission: create a standalone "confirmation corner" module that shows how you'll handle deletion warnings, loading states, and success feedback. This isn't the whole app—just the safety net. Make it so clear and friendly that your friend never panics again.

Objective

Produce a Flet application that implements a confirmation dialog for deleting a journal entry, a loading overlay for a simulated data save, and a snackbar for positive feedback. This project exercises AlertDialog, ProgressRing with overlay, and SnackBar, building components that will later integrate into the full journal app.

Creative Brief

Your friend loves the journal app you've been building together, but now they're worried: "What if I accidentally delete an entry? And sometimes the app feels like it's thinking without letting me know—can you add a spinner or something? Oh, and when I do something right, it should give me a little thumbs-up!" They want the app to feel safe and responsive. Your mission: create a standalone "confirmation corner" module that shows how you'll handle deletion warnings, loading states, and success feedback. This isn't the whole app—just the safety net. Make it so clear and friendly that your friend never panics again.

Materials and Resources

- A computer with Python and Flet installed (as set up in previous chapters)
- A code editor (VS Code, PyCharm, or any text editor)
- The Flet documentation on AlertDialog, ProgressRing, and SnackBar (optional quick reference)
- A stopwatch or timer to simulate a short async task (no external libraries needed)

Execution Steps

1. Set up a basic Flet page with a title and a few buttons

- Professional insider tip: Use page.title and page.horizontal_alignment to center your buttons for a clean look.
- Common mistake to avoid: Forgetting to import flet as ft at the top of your script.

2. Implement an AlertDialog that asks "Delete this entry?" when a delete button is pressed

- Professional insider tip: Set the dialog's modal property to True to force user interaction before continuing.
- Common mistake to avoid: Not passing the page to the dialog's init or forgetting to call page.dialog = dlg and dialog.open.

3. Add a loading overlay with a ProgressRing that appears when a "Save" button is clicked, then disappears after 2 seconds

- Professional insider tip: Wrap the overlay in a Container with a semi-transparent background color to dim the page behind it.
- Common mistake to avoid: Not using page.overlay.append() to add the overlay, or forgetting to remove it after the task completes.

4. Show a SnackBar with a success message after the loading overlay finishes

- Professional insider tip: Use snackbar.content = ft.Text("Your message") and set bgcolor to a calm green for positive feedback.
- Common mistake to avoid: Calling page.snack_bar = snackbar without also calling page.update() to refresh the UI.

5. Wire everything together: when the user confirms deletion in the dialog, trigger the loading overlay, then the snackbar

- Professional insider tip: Use async/await for the simulated delay to keep the UI responsive.
- Common mistake to avoid: Blocking the UI thread with time.sleep()—always use asyncio.sleep() inside an async function.

Self-Assessment Checklist

- The app displays a button labeled “Delete Entry” (or similar) that opens a modal AlertDialog.
- The AlertDialog includes a “Delete” action and a “Cancel” action, and only proceeds on “Delete”.
- When deletion is confirmed, a loading overlay with a ProgressRing appears and remains visible for at least 2 seconds.
- The overlay properly dims the background and prevents interaction with buttons behind it.
- After the loading overlay finishes, a SnackBar appears with a clear success message.
- The SnackBar disappears automatically after a few seconds or on user swipe.
- The app runs without errors and handles rapid button clicks gracefully (no double dialogs).
- The code uses appropriate Flet controls and follows the patterns taught in the chapter.

Bonus Challenge

Extend the app to include a list of mock journal entries, and make the delete button remove an entry from the list after user confirmation. Then, replace the simulated save with a real async operation that writes to a JSON file (using aiofiles, if installed). Add a “Restore” snackbar action that undoes the deletion.

Submission Format

Accepted formats:

- video (.mp4/.mov/.webm)

Minimum delivery:

- One video recording (up to 2 minutes) demonstrating your app: show the delete button, the confirmation dialog, the loading overlay, and the final snackbar in sequence. Narrate briefly what each step does and how you built it.

Development Guide

1. Research Phase
Look for examples of small apartment designs on sites like ArchDaily or Dezeen. Read news articles on micro-apartments to see both praise and criticism. Note how different designs handle the space-saving vs. comfort tradeoff. Use the course chapters on apartment planning and 3D modeling as your main guides.
2. Organization Phase
Start by stating your main argument — your position on balancing efficiency and comfort. Then outline three to four key points that support your view, using the mandatory references. For each point, plan what evidence or examples you'll use. Also think about one counterargument and how you'll respond to it.
3. Writing Phase
Write an introduction that presents the problem and your thesis. In the body paragraphs, each should focus on one key point — explaining a course concept and applying it to your design example. Use simple language: imagine explaining to a friend. Include specific examples, like how you'd place a bed or choose lighting. End with a conclusion that summarizes your argument and suggests practical tips for designers.
4. Review Phase
Check that your essay clearly states your position. Verify that you've used all four mandatory references correctly. Read it aloud to see if the logic flows smoothly. Ask yourself: would a beginner understand this? Remove any jargon or complex terms without explanation.

Self-Assessment Rubric

1. 1. Modal AlertDialog trigger button

- Insufficient — No delete button, or button does nothing, or dialog never appears.
- Adequate — Button opens a dialog, but it may not be strictly modal or lacks some styling.
- Excellent — Button clearly labeled, opens a perfectly modal dialog that dims background and blocks interaction.

2. 2. AlertDialog actions and behavior

- Insufficient — Missing actions, or any click immediately proceeds to loading without confirmation.
- Adequate — Buttons present but one may lack a handler or both behave identically.
- Excellent — Both Delete and Cancel actions present and correctly wired; Cancel dismisses, Delete triggers loading overlay.

3. 3. Loading overlay appearance and timing

- Insufficient — No overlay, or disappears immediately, or freezes app.
- Adequate — Overlay appears but may not cover page fully, ProgressRing missing, or delay < 2 s.
- Excellent — Overlay appears instantly, full-screen semi-transparent tint with a centered ProgressRing, stays for exactly 2 seconds, then removed.

4. 4. Overlay dimming and interaction blocking

- Insufficient — Overlay does not block interaction or is missing.
- Adequate — Overlay present but too transparent or not expanded, allowing accidental clicks.
- Excellent — Container with expand=True and semi-transparent color blocks all clicks underneath.

5. 5. SnackBar with success message

- Insufficient — No SnackBar after overlay.
- Adequate — SnackBar appears but with default style or vague message.
- Excellent — SnackBar slides in with a friendly, specific message, green background, visible and auto-dismisses.

6. 6. SnackBar auto-dismiss

- Insufficient — SnackBar never disappears without app restart.
- Adequate — SnackBar requires manual dismissal or stays too long.
- Excellent — SnackBar vanishes after ~4 seconds without user action, swipe-dismissible.

7. 7. Robustness (no errors, graceful rapid clicks)

- Insufficient — Crashes on rapid clicks or becomes unusable.
- Adequate — App works but rapid double-clicks may cause duplicate dialogs or stuck overlays.
- Excellent — No runtime errors; buttons disabled or checks prevent duplicate dialogs/overlays.

8. 8. Code quality and Flet patterns

- Insufficient — Non-Flet solutions, no async, copy-pasted code without understanding.
- Adequate — Code works but uses time.sleep (blocking), or missing overlay removal, or messy.
- Excellent — Clean, async/await used, proper overlay management (append/remove), page.update() called, all Flet controls correctly implemented.

Model Response & Assessment Reference

> *This guide presents possible paths, not single answers. An excellent essay may defend any of the theses below — or an original thesis not anticipated here — as long as it demonstrates argumentative rigor, correct use of sources, and autonomous critical reflection.*

1. Expected Thesis Position(s)

The central question admits multiple legitimate positions. Below are argumentative paths representing robust and distinct directions.

> ***Author note:*** *The recommended range is 2 to 4 theses. If the assignment has only one defensible answer, keep just Thesis A and adapt the wording. A third thesis often works best as a synthetic or dialectical position.*

1.1. Thesis A — {Short label}

{Full thesis statement, in one or two sentences. It should take a clear position on the central question.}

Main argumentative line: {One paragraph describing how this thesis sustains itself: the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.2. Thesis B — {Short label}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {One paragraph describing the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.3. Thesis C — {Synthetic/dialectical position, optional}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {Show explicitly how this position produces a new insight that neither A nor B alone can reach.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2. Key Concepts and Their Correct Use

> ***Author note:*** *Include 3 to 5 key concepts per project. For each concept: precise definition + example of correct application + common pitfall. The pitfall section is critical — it tells the evaluator what to watch for.*

2.1. A — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work — not just being named, but actively grounding a claim.}

Common pitfall: {The most frequent way students misuse this concept: over-application, decorative citation, missing nuance, ignoring critiques, or conflating related concepts. Be concrete.}

2.2. B — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2.3. C — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

3. Model Argumentative Structure

The outline below illustrates the structure of an excellent-level essay, regardless of which thesis the student chooses.

> ***Author note:*** *Word ranges are suggestions calibrated for a {total length}-word essay. Adjust proportionally if the assignment is shorter or longer.*

3.1. Introduction ({X}–{Y} words)

Contemporary hook: {Describe what kind of opening works for this project — a concrete example, striking data point, scene from culture, or current event. Adapt to the project's domain.}

Bridge to subject: {How the hook should pivot into the project's central question.}

Explicit thesis: The author's position in one or two clear, specific sentences, positioned directly against the central question.

3.2. Development ({X}–{Y} words)

Suggested: 2–3 paragraphs, each with a central argument.

Paragraph 1: {Function and how it advances the thesis.}

Paragraph 2: {Function and how it advances the thesis.}

Paragraph 3: {Function and how it advances the thesis.}

3.3. Counterargument ({X}–{Y} words)

An excellent essay does not merely mention an objection: it presents the strongest counterargument with intellectual honesty and responds consistently.

> ***Author note:*** *List the best counterargument for each thesis option, with a suggested response. The student should not feel that confronting the objection weakens their position — done well, it strengthens it.*

If thesis is A: {Strongest objection and possible response.}

If thesis is B: {Strongest objection and possible response.}

If thesis is C: {Strongest objection and possible response.}

3.4. Conclusion ({X}–{Y} words)

Restatement of thesis: Reaffirm the position in light of the full argumentation, without repeating the introduction verbatim.

Implications: {What this reflection reveals about something larger — the field, how we think, or a contemporary issue.}

Opening (no new information): A provocative question or forward-looking gesture that broadens the horizon without introducing arguments not developed in the essay.

3.5. Evidence and Data Types to Mobilize

- Specific passages from primary sources.
- Theoretical concepts with citation to the original work.
- Documented contemporary examples: news, official reports, data, or cultural productions.
- References to academic debates covered in the course.

4. Rubric — Excellent Level

Detailed description of what the evaluator should observe in an essay rated “Excellent” on each criterion.

> ***Author note:*** *The criteria below cover the standard dimensions of argumentative essays. Remove or adapt criteria that do not apply to the specific project.*

4.1. Thesis clarity

What is observed at the EXCELLENT level: The thesis appears in the introduction unequivocally. It is specific and directly engages the project's central tension. The entire essay is organized around it.

4.2. Use of course concepts

What is observed at the EXCELLENT level: Required references are defined precisely before application. Concepts are integrated organically into the argument, and their limits are recognized.

4.3. Quality of argumentation

What is observed at the EXCELLENT level: Each paragraph opens with a clear claim, sustains it with evidence, and connects back to the thesis. Logical progression is clear and the student analyzes rather than merely describes.

4.4. Counterarguments

What is observed at the EXCELLENT level: The student presents the strongest objection to their position and responds with a substantial argument. The thesis emerges stronger from confronting the critique.

4.5. Originality of analysis

What is observed at the EXCELLENT level: The essay goes beyond reproducing course materials. The student offers original connections, asks new questions, or proposes a novel interpretation from familiar evidence.

4.6. Connection to the contemporary

What is observed at the EXCELLENT level: The dialogue between subject and present is specific and grounded. Concrete contemporary examples illuminate both sides.

4.7. Structure and academic norms

What is observed at the EXCELLENT level: Cohesive structure with fluid transitions. Citations follow the established academic pattern, and references are complete and correctly formatted.

5. Common Argumentative Errors

The errors below are the most frequent in this type of project. The evaluator should watch for them.

> ***Author note:*** *Errors 2, 3, and 4 apply to almost any argumentative project. Errors 1 and 5 are project-specific slots where particular risks may be described.*

5.1. {Project-specific error}

How it appears in the text: {Verbatim-style example showing the error in action.}

How to correct: {Concrete, actionable advice on what to do differently.}

5.2. Strawman fallacy

How it appears in the text: The student presents the counterargument in its weakest, easiest-to-refute form instead of confronting the strongest version.

How to correct: Reformulate the counterargument as an intelligent interlocutor would present it, then respond to that reason.

5.3. Decorative use of concepts

How it appears in the text: The student names a concept but never applies it — it appears as a loose citation, disconnected from the argument.

How to correct: After citing a concept, explain exactly how it grounds the specific argument of that paragraph.

5.4. False equivalence

How it appears in the text: The student treats as equivalent two phenomena that operate in radically different contexts or functions.

How to correct: Make differences explicit before drawing parallels. Use precise comparative language. Comparison is not equation.

5.5. {Project-specific error}

How it appears in the text: {Example}

How to correct: {Correction}

> ***Final reminder:*** *The quality of reasoning is always more important than the position defended. An essay that defends an unpopular thesis with rigor outperforms one that defends a consensus thesis superficially.*